REPORT ASSIGNMENT

AYA MOHAMED
SEC :1
B.NO :9

Dr.Lamiaa 2024

LEAF _EXAMPLE IN SIMPLE CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

EXPLAIN WHY THERE ARE 2 CODES

-First Code is How to take input from the user by systemcalls and calc. the F value in the main and no need to call any function

-Second Code is also How to take input from the user by systemcalls and calc. the F value but in the leaf_example method

LEAF _EXAMPLE IN SIMPLE CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

THE CODE

Edit Execute

leafexample*

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : Leaf_Example



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in Leaf _Example\n"
number:       .asciiz  "Enter number\n"
result:    .asciiz "The Result is \n"
 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
   main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,number
  li $v0,4
  syscall

  li $v0,5
  syscall
```

THE CODE

Edit | Execute

leafexample*

```
    li $v0,4
    syscall

    li $v0,5
    syscall

    move $s0,$v0
    la $a0,number
    li $v0,4

    syscall
    li $v0,5
    syscall

    move $s1,$v0
    la $a0,number
    li $v0,4
    syscall

    li $v0,5
    syscall

    move $s2,$v0
    la $a0,number
    li $v0,4
    syscall
```

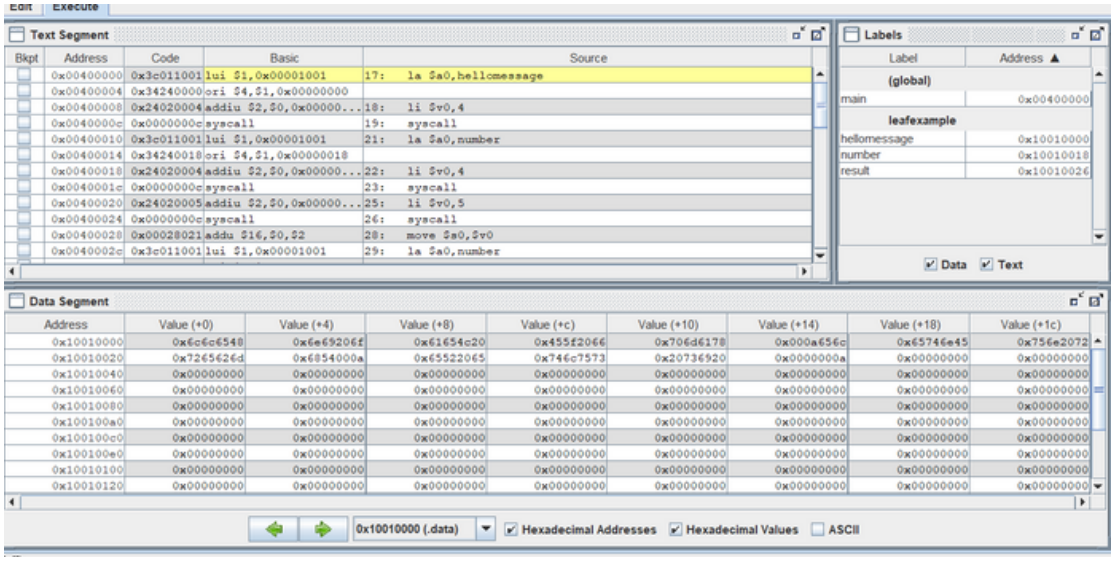
syscall Issue a system call : Execu

```

    move $s2,$v0
    la $a0,number
    li $v0,4
    syscall

    li $v0,5
    syscall

    move $s3,$v0
    add $t0,$s0,$s1
    add $t1,$s2,$s3
    sub $s4,$t0,$t1

    la $a0,result
    li $v0,4
    syscall

    move $a0,$s4
    li $v0,1
    syscall

    li $v0,10
    syscall
```

REGISTERS VALUES

$t1	9	0x00000003
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000004
$s1	17	0x00000005
$s2	18	0x00000001
$s3	19	0x00000002
$s4	20	0x00000006

VALUE IN MEMORY SEGMENT



SAMPLE OUTPUT & INPUT

```
Hello in Leaf _Example
Enter number
4
Enter number
5
Enter number
1
Enter number
2
The Result is
6
-- program is finished running --
```

LEAF _EXAMPLE IN SIMPLE CODE EXPLAIN THE CODE

This MIPS assembly code is a simple program called "Leaf_Example". Here's a breakdown of what it does:

1. Data Segment:
   - It declares three strings: hellomessage, number, and result. These strings are used for displaying messages to the user.
2. Code Segment:
   - The .text directive starts the code segment.
   - main:: This label marks the beginning of the main function.
   - It first prints a greeting message using the syscall with $v0 set to 4 (print string) and the address of hellomessage loaded into $a0.
   - Then, it prints a prompt for the user to enter a number using the same syscall mechanism.
   - Next, it reads an integer input from the user using syscall 5 (read integer) and stores it in register $v0.
   - It repeats the process for the next three numbers, storing them in registers $s0 to $s3.
   - After obtaining the four numbers, it adds the first two numbers and stores the result in $t0, adds the last two numbers and stores the result in $t1, and then subtracts the second sum from the first to get the final result, storing it in $s4.
   - It then prints the message "The Result is " using syscall 4 (print string) with the address of result.
   - The result stored in $s4 is printed using syscall 1 (print integer).
   - Finally, it exits the program by loading 10 into $v0 (syscall code for program exit) and executing syscall 10.

LEAF _EXAMPLE IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Leaf Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

THE CODE

Leafexample Function

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : Leaf_Example



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in Leaf _Example\n"
number:       .asciiz  "Enter number\n"
result:    .asciiz "The Result is \n"
 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
  main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,number
  li $v0,4
  syscall

  li $v0,5
  syscall
```

THE CODE

Leafexample Function

```
  li $v0,5
  syscall

  move $s0,$v0
  la $a0,number
  li $v0,4

  syscall
  li $v0,5
  syscall

  move $s1,$v0
  la $a0,number
  li $v0,4
  syscall

  li $v0,5
  syscall

  move $s2,$v0
  la $a0,number
  li $v0,4
  syscall

  li $v0,5
  syscall

```

```
  move $s3,$v0
  move $a0,$s0
  move $a1,$s1
  move $a2,$s2
  move $a3,$s3
jal   leaf_example

  la $a0,result
  li $v0,4
  syscall

  move $a0,$s0
  li $v0,1
  syscall

  li $v0,10
  syscall
```

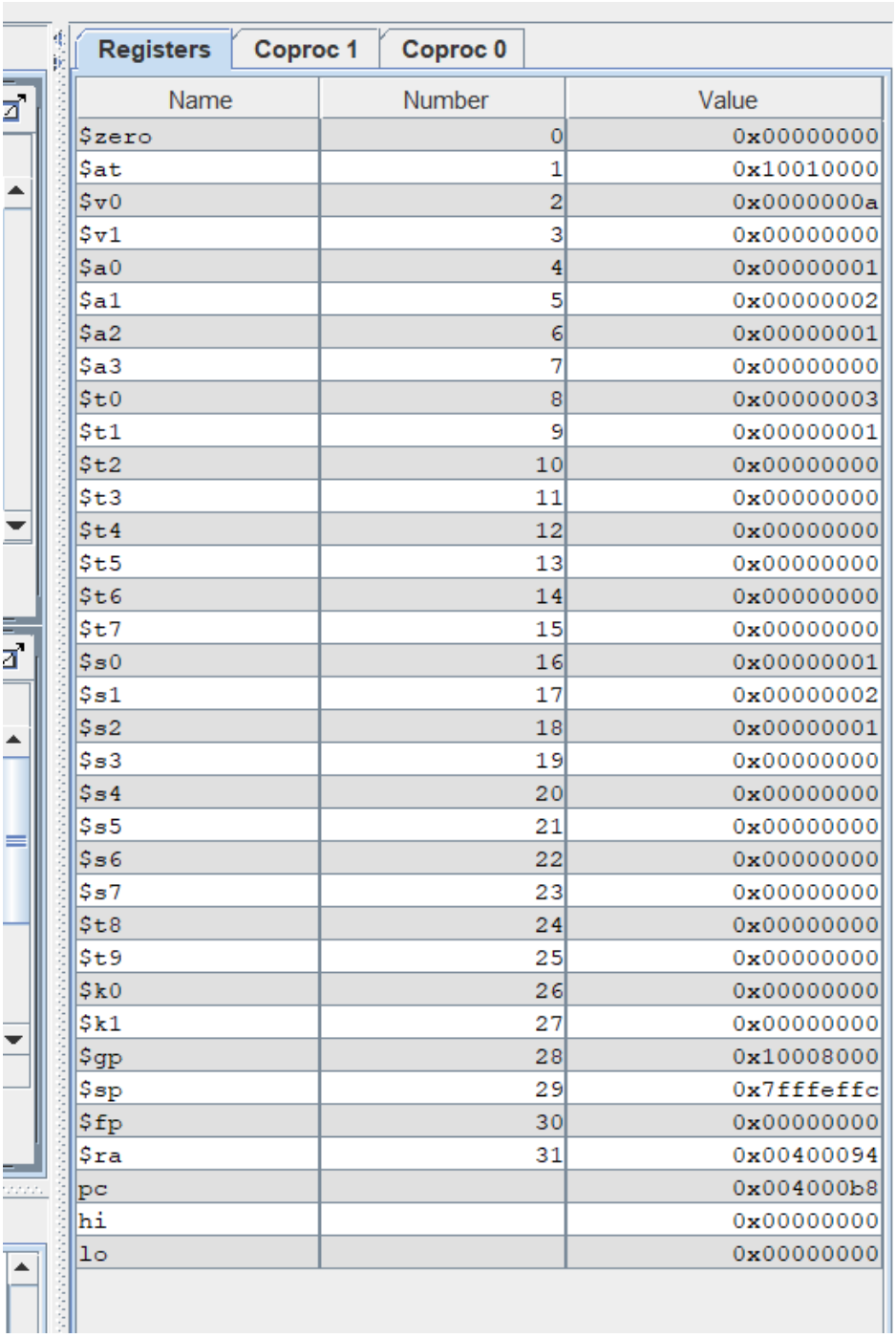
THE CODE OF FUNCTION

```
  move $a3,$s3
jal   leaf_example

  la $a0,result
  li $v0,4
  syscall

  move $a0,$s0
  li $v0,1
  syscall

  li $v0,10
  syscall
  leaf_example:
addi $sp, $sp, -4
sw $s0, 0($sp)
add $t0, $a0, $a1
add $t1, $a2, $a3
sub $s0, $t0, $t1
add $v0, $s0, $zero
lw $s0, 0($sp)
addi $sp, $sp, 4
jr $ra
```

SAMPLE OUTPUT & INPUT

```
Hello in Leaf _Example
Enter number
4
Enter number
5
Enter number
1
Enter number
2
The Result is
6
-- program is finished running --
```

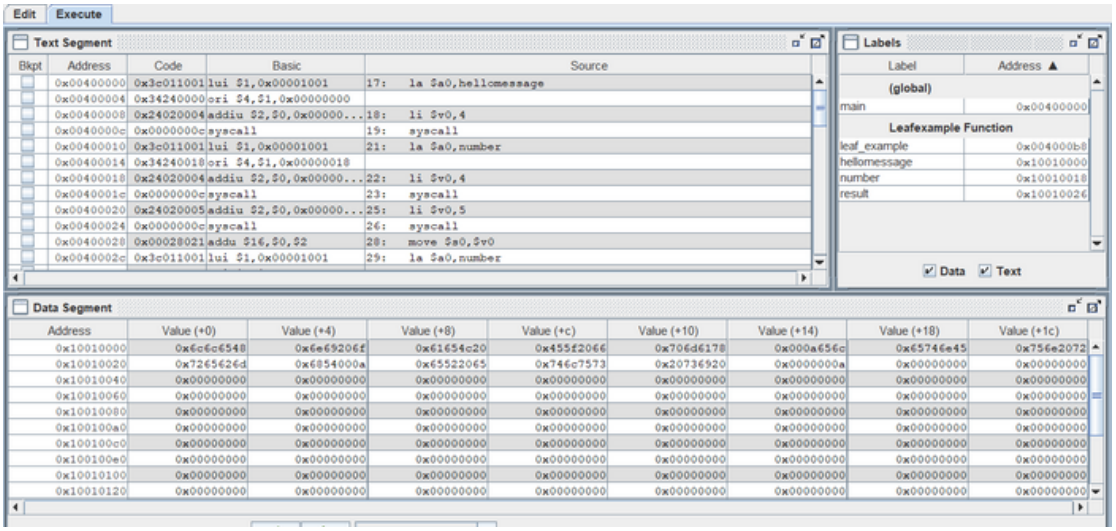
REGISTERS VALUES



Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x00000001
$a1	5	0x00000002
$a2	6	0x00000001
$a3	7	0x00000000
$t0	8	0x00000003
$t1	9	0x00000001
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000001
$s1	17	0x00000002
$s2	18	0x00000001
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00400094
pc		0x004000b8
hi		0x00000000
lo		0x00000000

VALUE IN MEMORY SEGMENT



LEAF _EXAMPLE IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code defines a program called "Leaf_Example" that consists of two parts: the `main` function and a separate function called `leaf_example`.

Here's a breakdown:

1. **Data Segment**:
   - Defines three strings: `hellomessage`, `number`, and `result`, used for displaying messages.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a prompt for the user to enter a number.
   - Reads the first number input by the user and stores it in register `$s0`.
   - Repeats the process for the next three numbers, storing them in registers `$s1` to `$s3`.
   - Then, it moves the contents of registers `$s0` to `$s3` into registers `$a0` to `$a3`, respectively, which are argument registers for function calls in MIPS.
   - Calls the `leaf_example` function using the `jal` (jump and link) instruction.
   - After returning from the function call, it prints the message "The Result is" and then prints the content of register `$s0`, which holds the result calculated in the `leaf_example` function.
   - Finally, it exits the program.

LEAF _EXAMPLE IN FUNCTION CODE EXPLAIN THE CODE

3. **`leaf_example` function**:
 - Begins with setting up the stack frame by decrementing the stack pointer (`$sp`) and saving the value of register `$s0` onto the stack.
 - Calculates the result by adding the first two arguments (`$a0` and `$a1`) and storing the sum in `$t0`, adding the last two arguments (`$a2` and `$a3`) and storing the sum in `$t1`, and then subtracting `$t1` from `$t0` and storing the result in `$s0`.
 - Copies the result to register `$v0`, which is used to return values from functions in MIPS.
 - Restores the value of `$s0` from the stack.
 - Adjusts the stack pointer back to its original position.
 - Jumps back to the calling function using the `$ra` register (return address).

F TO C TEMP. CONVERTER IN SIMPLE CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

EXPLAIN WHY THERE ARE 2 CODES

-First Code is How to take input from the user by systemcalls and calc. the C temp. value in the main and no need to call any function

-Second Code is also How to take input from the user by systemcalls and calc. the C value but in the leaf_example method

F TO C TEMP. CONVERTER IN SIMPLE CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

CODE

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : F To C Converter



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in F To C Temp _Example\n"
number:        .asciiz  "Enter number\n"
result:     .asciiz "The Result is \n"
delta: .float 32.0
scalar: .float 5.0
another_scalar: .float 9.0
 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
   main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,number
  li $v0,4
  syscall
```

THE CODE

```
li $v0,6
syscall


# Convert Fahrenheit to Celsius
  l.s $f1, delta
  l.s $f2, scalar
  l.s $f3, another_scalar

  sub.s $f0, $f0, $f1
  mul.s $f0, $f0, $f2
  div.s $f0, $f0, $f3

la $a0,result
li $v0 ,4
syscall
li $v0, 2
mov.s $f12, $f0 # Move contents of register $f0 to register $f12
syscall


li $v0,10
syscall
```

SAMPLE OUTPUT & INPUT

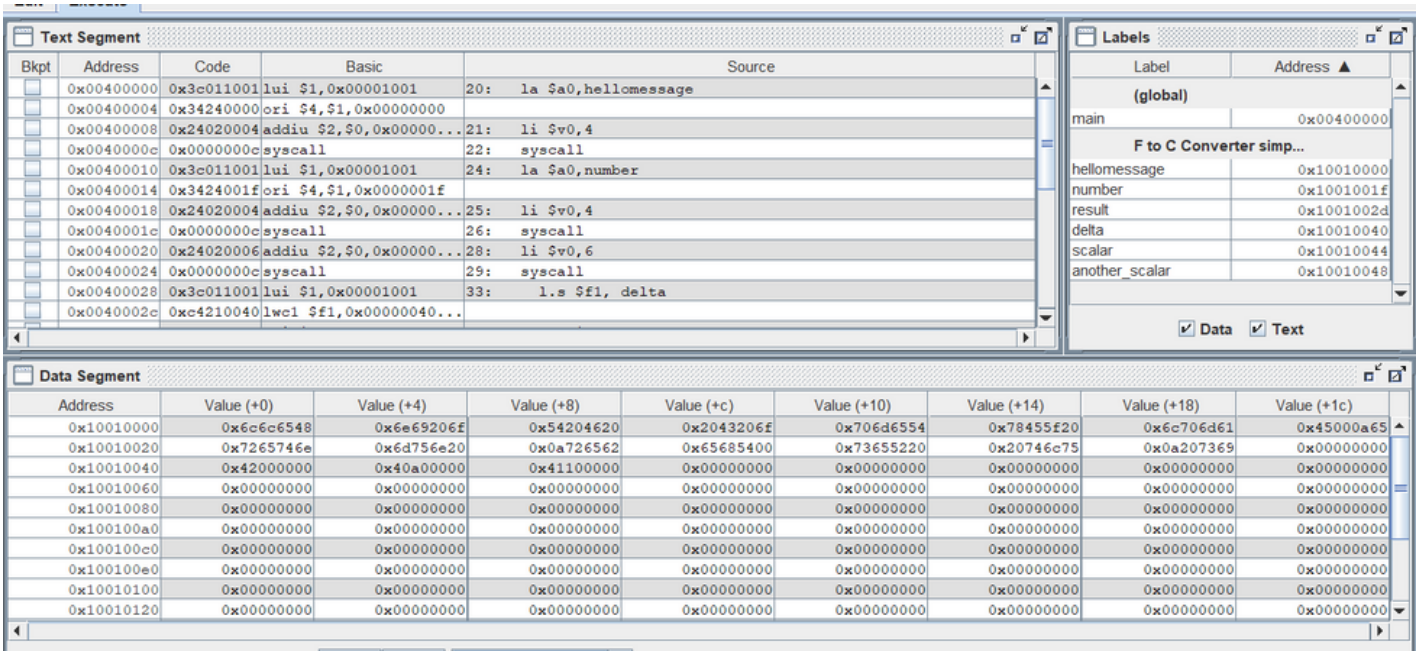
```
Hello in F To C Temp _Example
Enter number
75
The Result is
23.88889
-- program is finished running --
```

REGISTERS VALUES

$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400070
hi		0x00000000
lo		0x00000000

$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x1001002d

VALUE IN MEMORY SEGMENT



Text Segment

Bkpt	Address	Code	Basic		Source
	0x00400000	0x3c011001	lui $1,0x00001001	20:	la $a0,hellomessage
	0x00400004	0x34240000	ori $4,$1,0x00000000		
	0x00400008	0x24020004	addiu $2,$0,0x00000...	21:	li $v0,4
	0x0040000c	0x0000000c	syscall	22:	syscall
	0x00400010	0x3c011001	lui $1,0x00001001	24:	la $a0,number
	0x00400014	0x3424001f	ori $4,$1,0x0000001f		
	0x00400018	0x24020004	addiu $2,$0,0x00000...	25:	li $v0,4
	0x0040001c	0x0000000c	syscall	26:	syscall
	0x00400020	0x24020006	addiu $2,$0,0x00000...	28:	li $v0,6
	0x00400024	0x0000000c	syscall	29:	syscall
	0x00400028	0x3c011001	lui $1,0x00001001	33:	l.s $f1, delta
	0x0040002c	0xc4210040	lwc1 $f1,0x00000040...		

Labels

Label	Address ▲
(global)	
main	0x00400000
F to C Converter simp...	
hellomessage	0x10010000
number	0x1001001f
result	0x1001002d
delta	0x10010040
scalar	0x10010044
another_scalar	0x10010048

☑ Data ☑ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6c6c6548	0x6e69206f	0x54204620	0x2043206f	0x706d6554	0x78455f20	0x6c706d61	0x45000a65
0x10010020	0x7265746e	0x6d756e20	0x0a726562	0x65685400	0x73655220	0x20746c75	0x0a207369	0x00000000
0x10010040	0x42000000	0x40a00000	0x41100000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

F TO C TEMP. CONVERTER IN SIMPLE CODE EXPLAIN THE CODE

This MIPS assembly code is a program that converts Fahrenheit temperature to Celsius. Here's a breakdown of its functionality:

1. **Data Segment**:
   - Declares four strings: `hellomessage`, `number`, `result`, used for displaying messages to the user.
   - Defines two floating-point constants: `delta` with a value of 32.0 and `scalar` with a value of 5.0, and `another_scalar` with a value of 9.0. These constants are used in the conversion formula.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a prompt for the user to enter a number (Fahrenheit temperature).
   - Reads the Fahrenheit temperature input by the user using syscall 6 (read float).
   - Converts the Fahrenheit temperature to Celsius using the formula: `Celsius = (Fahrenheit - 32) * 5 / 9`.
   - Prints the message "The Result is" using syscall 4 (print string) with the address of `result`.
   - Prints the result (Celsius temperature) using syscall 2 (print float).
   - Exits the program.

The code achieves temperature conversion by performing floating-point arithmetic operations on the input Fahrenheit temperature (`$f0`) using the defined constants (`delta`, `scalar`, `another_scalar`) and MIPS floating-point instructions (`l.s`, `sub.s`, `mul.s`, `div.s`). Finally, it displays the result (Celsius temperature) to the user.

F TO C TEMP. CONVERTER IN FUNCTION CODE

Problem Statement:

-Convert The C program of the F to C temp. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

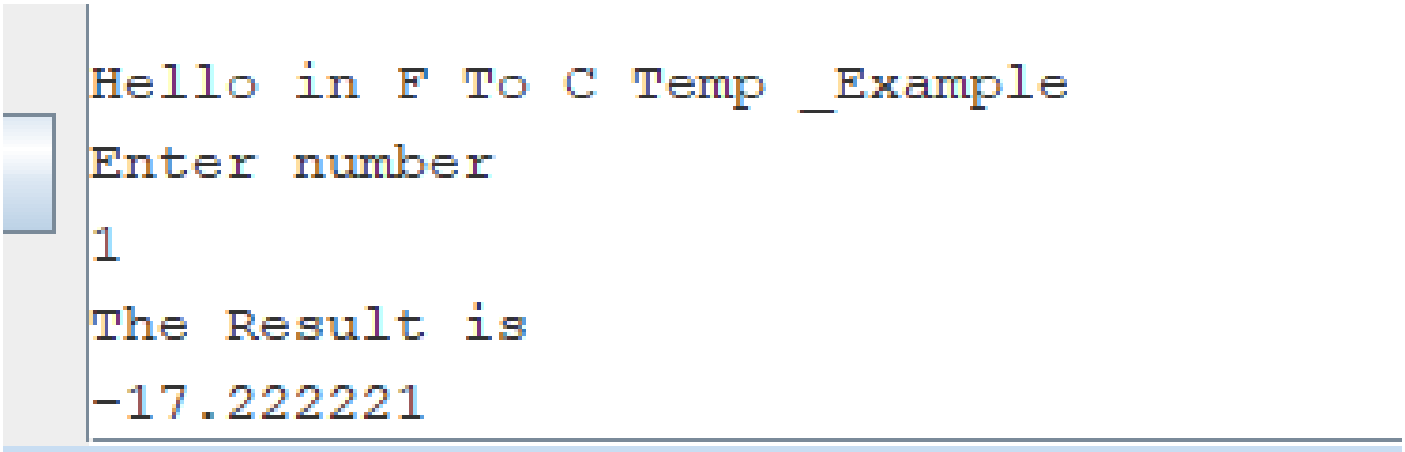
CODE

```
################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in F To C Temp _Example\n"
number:         .asciiz  "Enter number\n"
result:     .asciiz "The Result is \n"
delta: .float 32.0
scalar: .float 5.0
another_scalar: .float 9.0

 ################################################ Code Segment ################################################
.text
.globl main #Main function

main:
    la $a0,hellomessage
    li $v0,4
    syscall

    la $a0,number
    li $v0,4
    syscall

    li $v0,6
    syscall
```

THE CODE AND FUNCTION CODE

```
    # Save return address
    jal convertFtoC
    mov.s $f12, $f0       # Move Fahrenheit value to argument register for function call
    # Print the result
    la $a0,result
    li $v0 ,4
    syscall
    li $v0, 2
    syscall

    li $v0,10
    syscall
    # Function to convert Fahrenheit to Celsius
convertFtoC:


    l.s $f1, delta        # Load 32.0 into $f1
    l.s $f2, scalar       # Load 5.0 into $f2
    l.s $f3, another_scalar # Load 9.0 into $f3

    sub.s $f0, $f0, $f1   # Fahrenheit - 32
    mul.s $f0, $f0, $f2   # (Fahrenheit - 32) * 5
    div.s $f0, $f0, $f3   # (Fahrenheit - 32) * 5 / 9


    jr $ra                # Return to caller
```

SAMPLE OUTPUT & INPUT

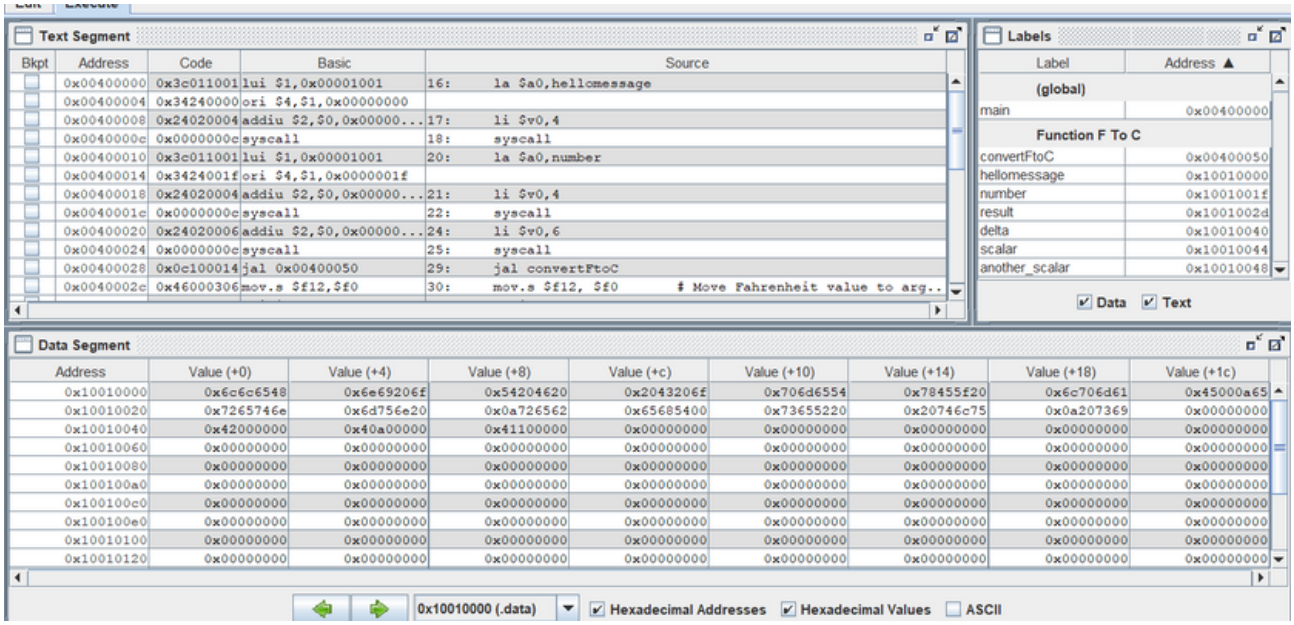


REGISTERS VALUES

$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x1001002d
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000000

$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x0040002c
pc		0x00400050
hi		0x00000000
lo		0x00000000

VALUE IN MEMORY SEGMENT



F TO C TEMP. CONVERTER IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code is a program that converts Fahrenheit temperature to Celsius using a function. Here's the breakdown of its functionality:

1. **Data Segment**:
   - Declares four strings: `hellomessage`, `number`, `result`, used for displaying messages to the user.
   - Defines three floating-point constants: `delta` with a value of 32.0, `scalar` with a value of 5.0, and `another_scalar` with a value of 9.0. These constants are used in the conversion formula.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a prompt for the user to enter a number (Fahrenheit temperature).
   - Reads the Fahrenheit temperature input by the user using syscall 6 (read float).
   - Calls the function `convertFtoC` to perform the Fahrenheit to Celsius conversion.
   - Moves the result (Celsius temperature) from the return value register `$f0` to `$f12`, the argument register for syscall 2 (print float).
   - Prints the message "The Result is" using syscall 4 (print string) with the address of `result`.
   - Prints the Celsius temperature using syscall 2 (print float).
   - Exits the program.

3. **`convertFtoC` function**:
   - This function is called from the `main` function.
   - Loads the floating-point constants `delta`, `scalar`, and `another_scalar` from memory.
   - Calculates the Celsius temperature using the formula: `Celsius = (Fahrenheit - 32) * 5 / 9`.
   - Returns the result by jumping back to the caller using the `$ra` register.

Overall, the code effectively converts Fahrenheit temperature to Celsius using a separate function.

COPY STRING IN FUNCTION CODE

Problem Statement:

-Convert The C program of the copy string. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

CODE

Leafexample Function | F to C Converter simpler code without FUNCTION | Function F To C | String copy

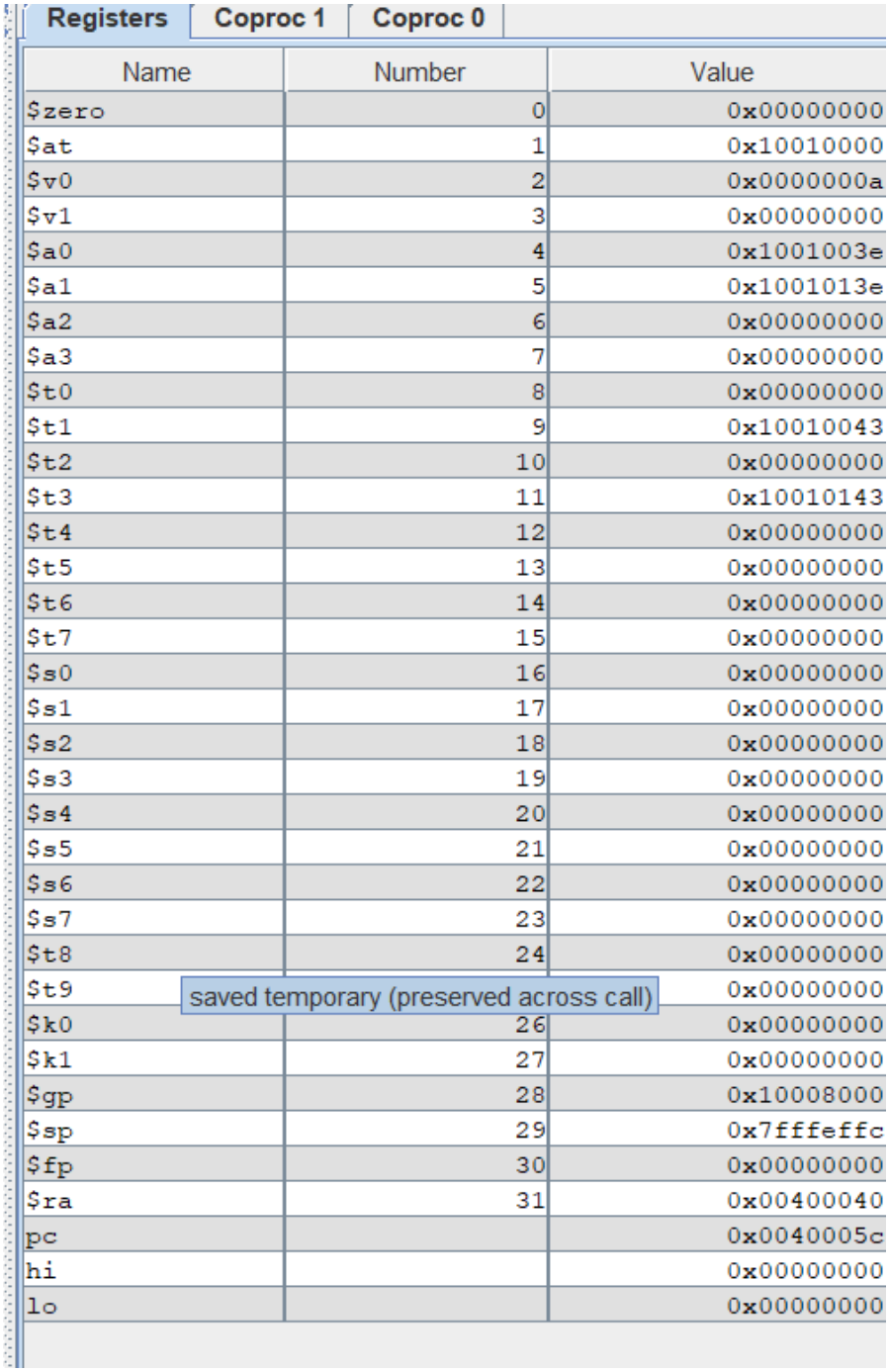
```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : String copy_Example



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in String copy _Example\n"
string:        .asciiz  "Enter String \n"
result:    .asciiz "The Result is \n"
buffer1: .space 256    # Allocate space for the input string
buffer2: .space 256    # Allocate space for the input string
 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
   main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,string
  li $v0,4
  syscall

  li $v0,8
```

THE CODE

```
28      la $a0, buffer1      # load address of buffer
29      li $a1, 256          # maximum number of characters to read
30    syscall
31    la $a1,buffer2
32  jal   stringcopy_function
33
34  move $a0,$a1
35
36
37      # Print the string
38      li $v0, 4            # syscall code for print_string
39      la $a0, buffer1       # load address of buffer
40      syscall
41
42      # Exit program
43      li $v0, 10           # syscall code for exit
44      syscall
45    stringcopy_function:
46
47  addi $sp, $sp, -4 # adjust stack for 1 item
48  sw $s0, 0($sp) # save $s0
49  add $s0, $zero, $zero # i = 0
50  L1: add $t1, $s0, $a0 # addr of y[i] in $t1
51  lbu $t2, 0($t1) # $t2 = y[i]
52  add $t3, $s0, $a1 # addr of x[i] in $t3
53  sb $t2, 0($t3) # x[i] = y[i]
54  beq $t2, $zero, L2 # exit loop if y[i] == 0
```

```
stringcopy_function:

addi $sp, $sp, -4 # adjust stack for 1 item
sw $s0, 0($sp) # save $s0
add $s0, $zero, $zero # i = 0
L1: add $t1, $s0, $a0 # addr of y[i] in $t1
lbu $t2, 0($t1) # $t2 = y[i]
add $t3, $s0, $a1 # addr of x[i] in $t3
sb $t2, 0($t3) # x[i] = y[i]
beq $t2, $zero, L2 # exit loop if y[i] == 0
addi $s0, $s0, 1 # i = i + 1
j L1 # next iteration of loop
L2: lw $s0, 0($sp) # restore saved $s0
addi $sp, $sp, 4 # pop 1 item from stack
jr $ra # and return
```

REGISTERS VALUES



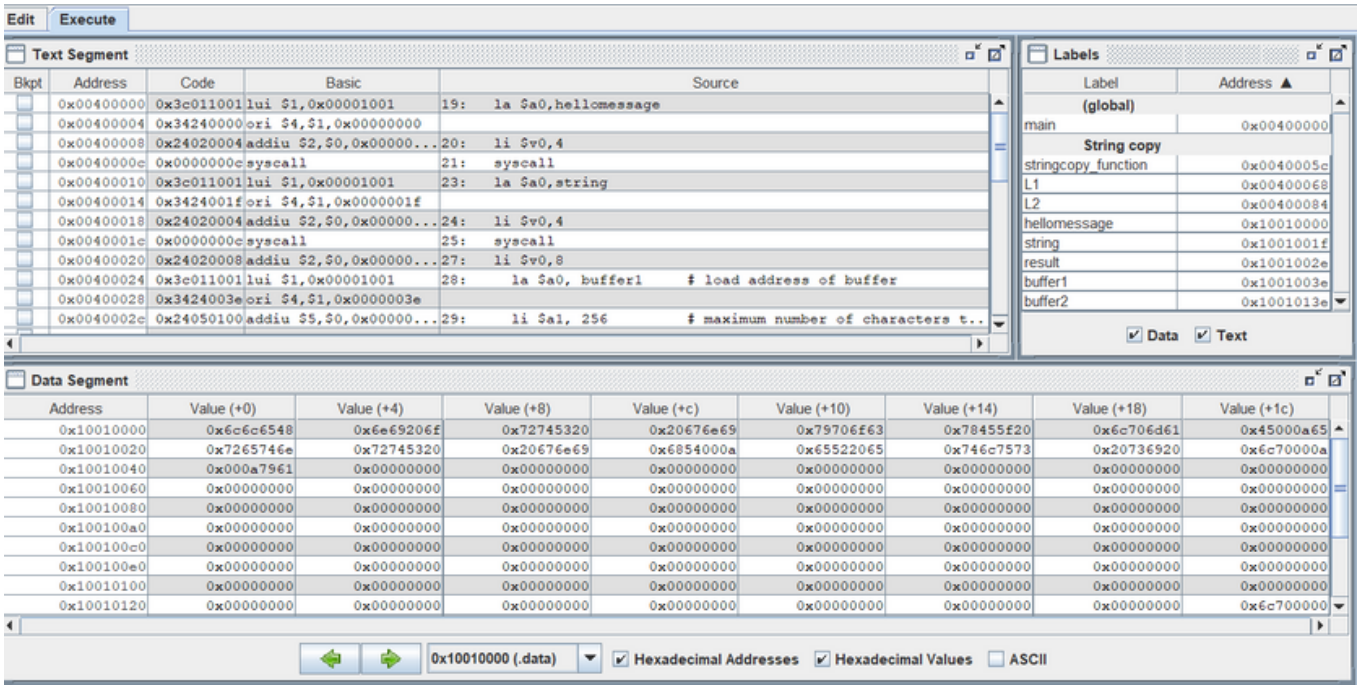
Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x1001003e
$a1	5	0x1001013e
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000000
$t1	9	0x10010043
$t2	10	0x00000000
$t3	11	0x10010143
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	saved temporary (preserved across call)	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00400040
pc		0x0040005c
hi		0x00000000
lo		0x00000000

SAMPLE OUTPUT & INPUT

```
Hello in String copy _Example
Enter String
play
play
```

VALUE IN MEMORY SEGMENT



COPY STRING IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code is a program that copies a string from one buffer to another buffer. Here's the breakdown of its functionality:

1. **Data Segment**:
   - Declares three strings: `hellomessage`, `string`, and `result`, used for displaying messages to the user.
   - Defines two buffers: `buffer1` and `buffer2`, each allocated with space to hold 256 characters. These buffers are used for storing the input string and the copied string.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a prompt for the user to enter a string.
   - Reads a string input by the user using syscall 8 (read string) into `buffer1`.
   - Calls the function `stringcopy_function` to copy the string from `buffer1` to `buffer2`.
   - Prints the copied string stored in `buffer1`.
   - Exits the program.

3. **`stringcopy_function` function**:
   - This function is called from the `main` function.
   - Copies the string from `buffer1` to `buffer2`.
   - It uses a loop to iterate through each character of the string stored in `buffer1`.
   - For each character, it loads it into a temporary register `$t2`, then stores it in the corresponding position in `buffer2`.
   - The loop continues until it encounters a null terminator (end of string).
   - After copying, it returns to the caller.

Overall, the code effectively demonstrates string copying functionality in MIPS assembly, using a separate function to perform the task.

FACTORIAL IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Factorial. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the User by Keybaord By systemcalls

Sample Output :

for each input by the user the respond output

CODE

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : Factorial_Example



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in Factorial _Example\n"
number:         .asciiz  "Enter number\n"
result:     .asciiz "The Result is \n"
 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
   main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,number
  li $v0,4
  syscall

  li $v0,5
  syscall
```

THE CODE

```
  move $a0,$v0
jal Fact
   move $s0,$v0


  la $a0,result
  li $v0,4
  syscall

  move $a0,$s0
  li $v0,1
  syscall

  li $v0,10
  syscall
  Fact:
  fact:
addi $sp, $sp, -8 # adjust stack for 2 items
sw $ra, 4($sp) # save return address
sw $a0, 0($sp) # save argument
slti $t0, $a0, 1 # test for n < 1
beq $t0, $zero, L1
addi $v0, $zero, 1 # if so, result is 1
addi $sp, $sp, 8 # pop 2 items from stack
jr $ra # and return
L1: addi $a0, $a0, -1 # else decrement n
jal fact # recursive call
lw $a0, 0($sp) # restore original n
lw $ra, 4($sp) # and return address
addi $sp, $sp, 8 # pop 2 items from stack
mul $v0, $a0, $v0 # multiply to get result
jr $ra # and return
```

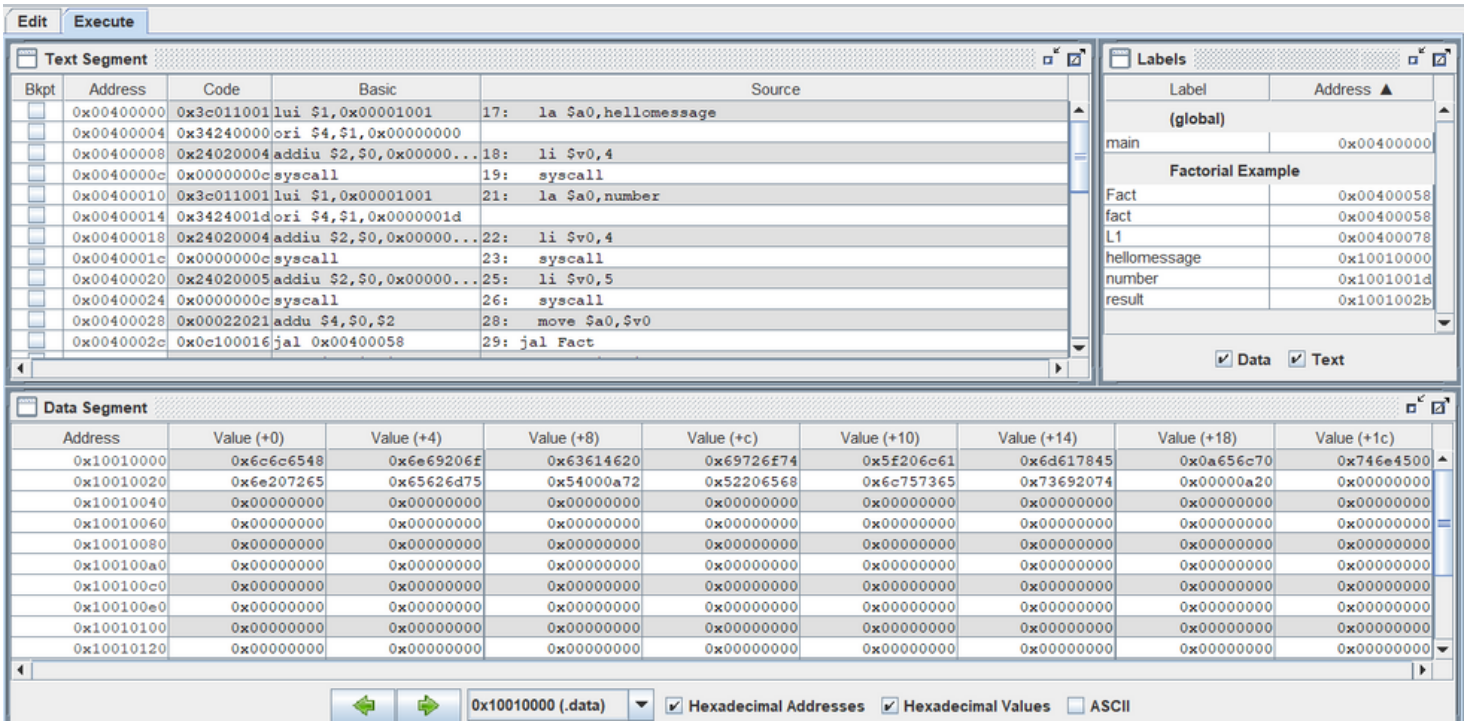
REGISTERS VALUES

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x00000078
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000001
$t1	9	0x00000000
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000078
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00400030
pc		0x00400058
hi		0x00000000
lo		0x00000078

SAMPLE OUTPUT & INPUT

```
Hello in Factorial _Example
Enter number
5
The Result is
120
```

VALUE IN MEMORY SEGMENT



FACTORIAL IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code is a program that calculates the factorial of a given number using recursion. Here's the breakdown of its functionality:

1. **Data Segment**:
   - Declares three strings: `hellomessage`, `number`, and `result`, used for displaying messages to the user.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a prompt for the user to enter a number.
   - Reads an integer input by the user using syscall 5 (read integer).
   - Calls the function `Fact` to calculate the factorial of the input number.
   - Stores the result (factorial) in register `$s0`.
   - Prints the message "The Result is" using syscall 4 (print string) with the address of `result`.
   - Prints the factorial stored in register `$s0` using syscall 1 (print integer).
   - Exits the program.

3. **`Fact` function**:
   - This function calculates the factorial of a given number recursively.
   - It starts with a label `fact` and saves the return address and argument on the stack.
   - It checks if the input number is less than 1. If true, it returns 1 as the factorial value.
   - If the input number is greater than or equal to 1, it decrements the input number and makes a recursive call to itself (`fact`).
   - After the recursive call returns, it multiplies the input number with the returned factorial value to calculate the factorial of the original input number.
   - Finally, it returns the factorial value.

SORT IN FUNCTION CODE

Problem Statement:

-Convert The C program of the Sort. Example to Assemply Lang. By Mars

Sample Input :

I take The input numbers from the array defined in the data segement

Sample Output :

is the sorted array

CODE

```
#Name : Aya Mohamed Sayed Hamed
#Sec : 1
#B.NO : 9
#Programa 1 : Sort_Example



################################################ Data Segment ################################################
.data
hellomessage: .asciiz "Hello in Sort _Example\n"
spacemessage: .asciiz "  "
string:         .asciiz  "The array stored in the registers is \n"
array_print:     .asciiz " 8 1 6 0 9 \n"
result:     .asciiz "The Result after sorting is \n"
array: .word 8, 1, 6, 0, 9   # Example array to be sorted
array_size: .word 5               # Size of the array

 ################################################ Code Segment ################################################
   .text
   .globl main #Main function
   main:
  la $a0,hellomessage
  li $v0,4
  syscall

  la $a0,string
  li $v0,4
```

THE CODE

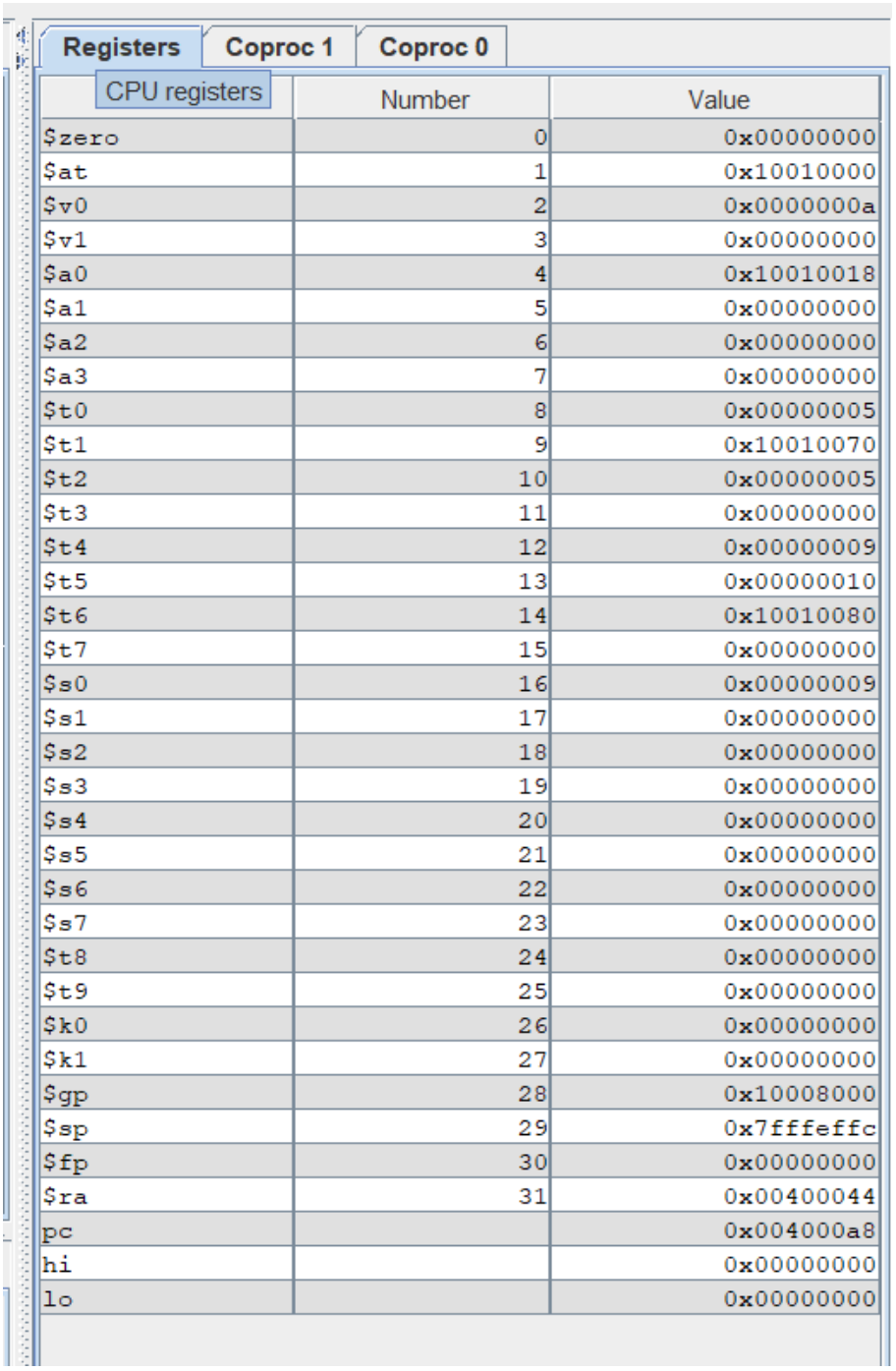
NOTE:
THE SORTED FUNCTION IS TAKEN FROM THE BOOK AS YOUR INSTRUCTION

```
la $a0,string
li $v0,4
syscall

la $a0,array_print
li $v0,4
syscall
   la $a0, array               # Load address of array
  lw $a1, array_size          # Load array size
  jal Sort
    la $a0,result
li $v0,4
syscall
  li $t0,0

  la $t1,array
  lw $t2,array_size

```

THE CODE FOR PRINT THE ARRAY AFTER SORTING

```
printarray:
    slt $t3, $t0, $t2
    beq $t3, $zero, exit_printarray
    sll $t5, $t0, 2
    add $t6, $t5, $t1
    lw $s0, 0($t6)
    move $a0, $s0
    li $v0, 1
    syscall
      la $a0,spacemessage
  li $v0,4
  syscall


    addi $t0, $t0, 1
    j printarray

exit_printarray:
    li $v0, 10
    syscall
```

REGISTERS VALUES



Registers | Coproc 1 | Coproc 0

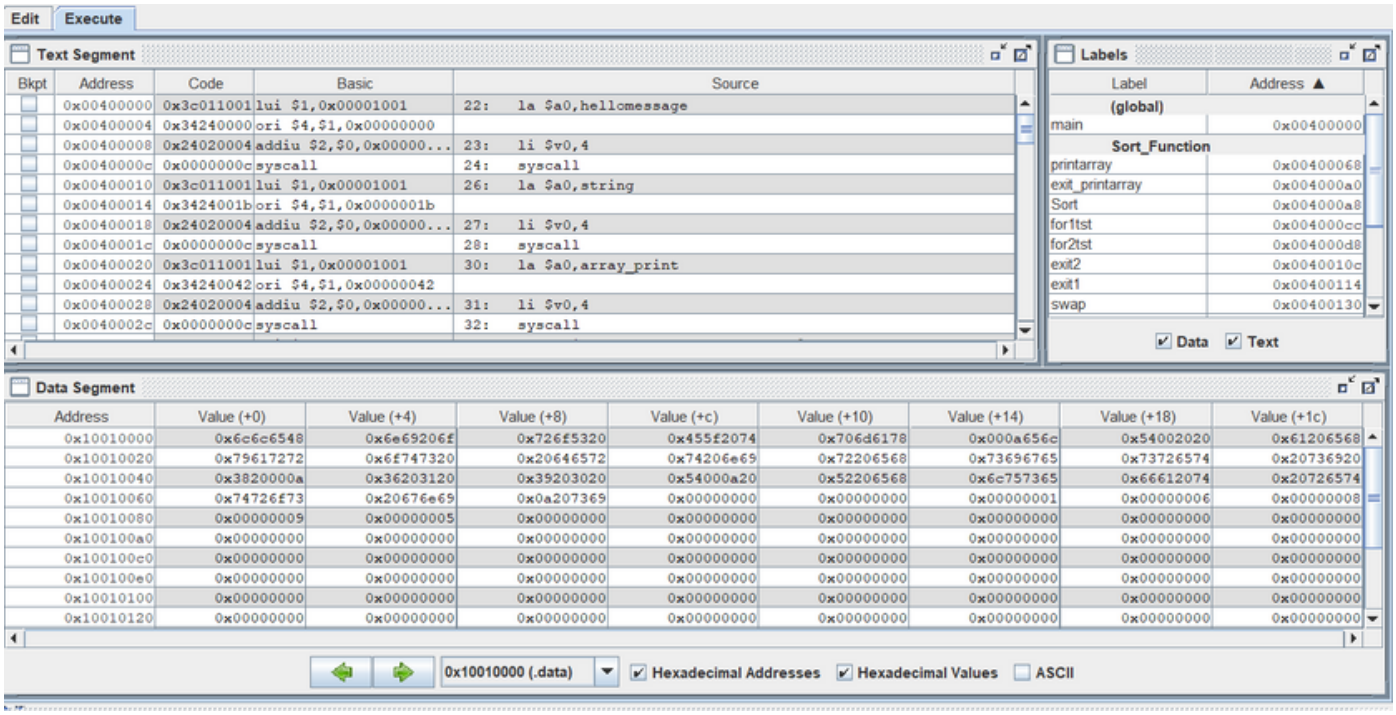
CPU registers

	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x10010018
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000005
$t1	9	0x10010070
$t2	10	0x00000005
$t3	11	0x00000000
$t4	12	0x00000009
$t5	13	0x00000010
$t6	14	0x10010080
$t7	15	0x00000000
$s0	16	0x00000009
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00400044
pc		0x004000a8
hi		0x00000000
lo		0x00000000

SAMPLE OUTPUT & INPUT

```
Hello in Sort _Example
The array stored in the registers is
 8 1 6 0 9
The Result after sorting is
0  1  6  8  9
```

VALUE IN MEMORY SEGMENT



SORT IN FUNCTION CODE EXPLAIN THE CODE

This MIPS assembly code performs sorting on an array using the bubble sort algorithm. Let's break down the code step by step:

1. **Data Segment**:
   - Declares five strings: `hellomessage`, `spacemessage`, `string`, `array_print`, and `result`, which are used for displaying messages to the user.
   - Declares an array `array` with initial values to be sorted.
   - Declares a word `array_size` to store the size of the array.

2. **Code Segment**:
   - `.globl main`: Marks the beginning of the `main` function.
   - Prints a greeting message and a message indicating the array stored in the registers.
   - Prints the content of the array using syscall 4.
   - Calls the `Sort` function to sort the array.
   - Prints a message indicating the sorted array.
   - Prints the sorted array using syscall 1.
   - Exits the program using syscall 10.

3. **`Sort` function**:
   - Begins with a label `Sort`.
   - Allocates space on the stack for 5 registers.
   - Saves the current register values onto the stack.
   - Initializes registers `$s0`, `$s1`, `$s2`, `$s3`, and `$ra` with values passed as arguments (`$a0` and `$a1`).
   - Starts a loop labeled `for1tst` to iterate over each element of the array.
   - Inside the loop, another loop labeled `for2tst` compares adjacent elements and swaps them if they are in the wrong order.
   - The `swap` function is called to perform the swapping operation.
   - Once the outer loop completes, the sorted array is returned.

4. **`swap` function**:
   - Swaps the values of two elements in the array.